REPORTE

Alumno :Luis Lara Andrade

Profesor :Victor Uc Cetina

1. Baja mas el error al anadir dos capas, o se estanco / empeoro? Ocurre igual en NumPy y en Keras?

Al anadir dos capas ocultas intermedias (pasando de 4x3x3 a 4x3x3x3x3), el error no bajo al contrario, se estanco severamente.

En la red original de dos capas, ambas implementaciones reducen su costo de manera continua durante las 500 epocas (en Keras, por ejemplo, el error desciende de 0.2255 hasta 0.1433). Sin embargo, en la arquitectura de cuatro capas, la perdida apenas disminuye unas cuantas milesimas y la curva queda practicamente plana cerca de 0.20 a 0.22.

Lo mismo ocurre de forma identica en NumPy como en Keras, lo que comprueba que no se trata de un error de sintaxis, sino de una limitacion del entrenamiento de redes mas profundas bajo estas condiciones.

2.Las curvas de la notebook 01 (NumPy) y de Keras se parecen con la misma topologia? Si no, que diferencias de implementacion podrian explicarlo?

Las curvas siguen la misma tendencia general (descenso sostenido en la red de 2 capas y estancamiento en la de 4 capas), pero presentan diferencias claras en su forma y velocidad de convergencia:

- Regimen de actualizacion y tamano de lote (batch size): En NumPy el entrenamiento se programo estrictamente en modo estocastico u online (batch_size = 1), actualizando los pesos muestra por muestra (150 actualizaciones por epoca sobre los datos sin barajar). Keras, por defecto, agrupa los datos en minilotes de 32 ejemplos (5 actualizaciones por epoca), produciendo un descenso mucho mas suave y amortiguado.
- Inicializacion de pesos: En NumPy se utilizo una inicializacion uniforme simple np.random.rand() - 0.5, mientras que Keras utiliza por defecto GlorotUniform (), la cual escala la varianza en funcion del numero de entradas y salidas de cada capa, alterando la posicion de partida en la superficie de perdida.

3. **Con sigmoides apiladas y MSE, tiene sentido que una red mas profunda no aprenda mejor en Iris? Relacionalo con las graficas.**

Tiene mucho sentido, y las graficas lo reflejan debido a 2 factores:

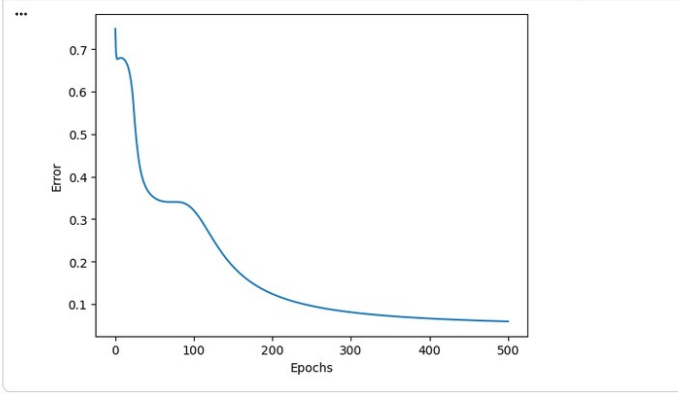
- Saturacion con funcion de costo MSE: A diferencia de funciones probabilisticas como categorical cross-entropy, el error cuadratico medio no penaliza con fuerza cuando una neurona sigmoide se satura cerca de 0 o 1 con prediccion erronea, debilitando aun mas la magnitud del gradiente.
- Simplicidad de Iris: El problema cuenta con solo 150 ejemplos y es linealmente separable en su mayoria. Una sola capa oculta de 3 neuronas ya tiene suficiente capacidad de representacion para trazar las

fronteras de decision; agregarle capas redundantes solo anade atenuacion de gradiente sin aportar ventaja verdadera.

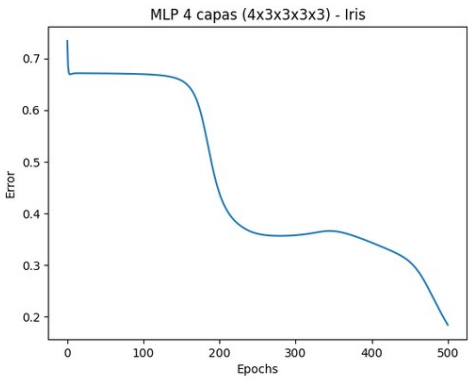
Evidencias

04

```
plt.xlabel('Epochs')
plt.ylabel('Error')
plt.show()
```




```
Error_vec = np.asarray(E)
plt.plot(Error_vec)
plt.xlabel('Epochs')
plt.ylabel('Error')
plt.title('MLP 4 capas (4x3x3x3x3) - Iris')
plt.show()
```



05

```
model.summary()
```

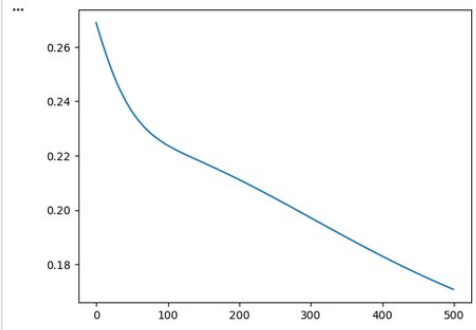
```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `in
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12

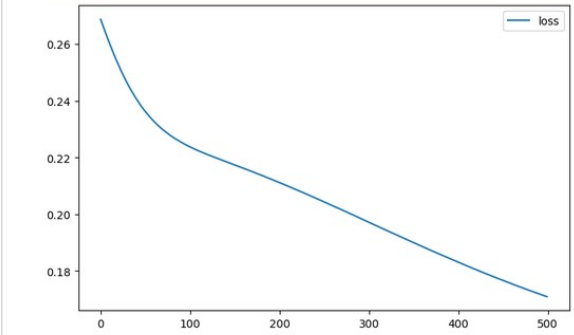
```
Total params: 27 (108.00 B)
Trainable params: 27 (108.00 B)
Non-trainable params: 0 (0.00 B)
```

```
y = model(X)
```

```
plt.plot(history.history['loss'])
plt.show()
```




```
pd.DataFrame(history.history).plot(figsize=(8,5))
plt.show()
```



```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `i
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential 1"

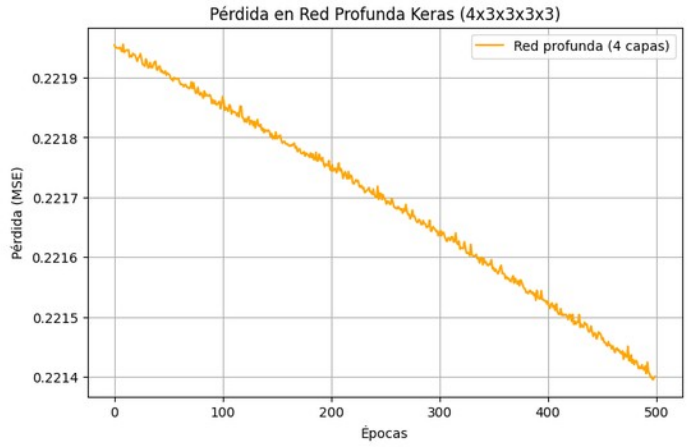
Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12
layer3 (Dense)	(None, 3)	12
layer4 (Dense)	(None, 3)	12

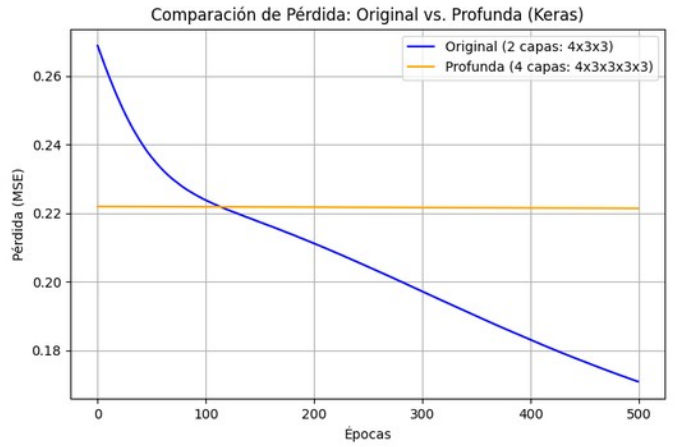
Total params: 51 (204.00 B)
Trainable params: 51 (204.00 B)
Non-trainable params: 0 (0.00 B)

```
y_deep = model_deep(X)
```

[15] ✓ 0s

```
plt.grid(True)
plt.show()
```





[16] ✓ 0s

```
model_deep.predict(np.array([[3, 3, 1, 1]]))
```